

Machine Learning

Instructor: Hafiz Abdul Rehman

Lecture 03: Evaluate Hypothesis

Supervised Learning

$$D = \{ x_1, y_1, x_2, y_2, \dots, x_n, y_n \} \subseteq X \times Y$$

- Where,
 - D is the data set
 - X is the d -dimensional feature space ($\mathbb{R}^d$)
 - x_i is the input vector of the i th sample
 - Y is the label space
- The data points are drawn from an unknown distribution P
$$(\vec{x}_i, y_i) \sim P(x, y)$$
- We want to learn a function $h \in H$, such that for a new instance $(\vec{x}_i, y_i) \sim P$
$$h(\vec{x}_i) = y$$
- With a high probability or at least, $h(\vec{x}_i) \sim y$

How to choose h ?

- Randomly
 - May not work well
 - Like using a random program to solve your sorting problem
 - May work if H is constrained enough
- Exhaustively
 - Would be very slow
 - The space H is usually very large (if not infinite)
- H is usually chosen by data scientists (you!) based on their experience!
 - $h \in H$ is estimated efficiently using various optimization techniques

How to evaluate h ?

Loss functions

- Calculate the average error of h in predicting y .
- Smaller is better
 - 0 loss: No error
 - 100% loss: Could not even get one instance right
 - 50% loss: Your h is as informative as a coin toss

Loss Function

0/1 Loss

$$L_{0/1}(h) = \frac{1}{n} \sum_{i=1}^n \delta_{h(x_i) \neq y_i}, \quad \text{Where } \delta_{h(x_i) \neq y_i} = \begin{cases} 1, & \text{if } h(x_i) \neq y_i \\ 0, & \text{otherwise} \end{cases}$$

- Counts the average number of mistakes in predicting y
- Returns the training error rate
- Non-continuous and non-differentiable
 - Difficult to utilize in optimization
- Used to evaluate classifiers in binary/multiclass settings

Squared loss

$$L_{sq}(\mathbf{h}) = \frac{1}{n} \sum_{i=1}^n (\mathbf{h}(\mathbf{x}_i) - \mathbf{y}_i)^2$$

- Typically used in regression settings
- The loss is always non-negative
- The loss grows quadratically with the absolute magnitude of mis-prediction
- Encourages no predictions to be really far off
- If a prediction is very close to be correct, the square will be tiny and little attention will be given to that example to obtain zero error

Absolute loss

$$L_{abs}(\mathbf{h}) = \frac{1}{n} \sum_{i=1}^n | \mathbf{h}(\mathbf{x}_i) - y_i |$$

- Typically used in regression settings
- The loss is always non-negative
- The loss grows linearly with the absolute magnitude of mis-prediction
- Better suited for noisy data

Comparison

y	h(x)	Square loss	Abs loss
100.00	101.00	1.00	1.00
90.00	91.00	1.00	1.00
100.00	101.00	1.00	1.00
20.00	21.00	1.00	1.00
30.00	29.00	1.00	1.00
40.00	41.00	1.00	1.00
30.00	31.00	1.00	1.00
10.00	11.00	1.00	1.00
12.00	13.00	1.00	1.00
16.00	17.00	1.00	1.00
1000.00	1,00.00	810,000.00	900.00
Overall		73,637.27	82.73

y	h(x)	Square loss	Abs loss
100.00	0.00	10,000.00	100.00
90.00	0.00	8,100.00	90.00
100.00	0.00	10,000.00	100.00
20.00	0.00	400.00	20.00
30.00	0.00	900.00	30.00
40.00	0.00	1,600.00	40.00
30.00	0.00	900.00	30.00
10.00	0.00	100.00	10.00
12.00	0.00	144.00	12.00
16.00	0.00	256.00	16.00
1,000.00	1,000.00	0.00	0.00
Overall		2,945.45	40.73

y	h(x)	Square loss	Abs loss
100.00	101.00	1.00	1.00
90.00	90.01	0.0001	0.01
100.00	200.00	10,000.00	100.00
100.00	1,000.00	810,000.00	900.00
Overall		205,000.25	250.25

The elusive h

$$h = \operatorname{argmin}_{h \in H} L(h)$$

So we need an h with a low loss on D ?

How not to reduce the loss?

$$h(\mathbf{x}) = \begin{cases} \mathbf{y}_i, & \text{if } \exists (\mathbf{x}_i, \mathbf{y}_i) \in D, \\ 0, & \text{otherwise} \end{cases} \quad \text{s.t. } X = \mathbf{x}_i$$

- What would be the loss of this h on the training set?
- What would be the loss of this h on an unseen test set?
- Why is it bad? **The memorizer!**
- How to prevent this from happening?

Generalization

$$\epsilon = E_{x,y \sim P}[l(x, y)|h]$$

- That the expected loss should be calculated on any data point sampled from the distribution P , not necessarily those present in D
- How to get a new datapoint $(x, y) \sim P$?
 - All we have are the n data points!
- We estimate ϵ by splitting the D :



- We train on D_{TR} and test on D_{TE} only **once!**
- Don't train on test inadvertently! (e.g. repeated testing)

Training and Test Data

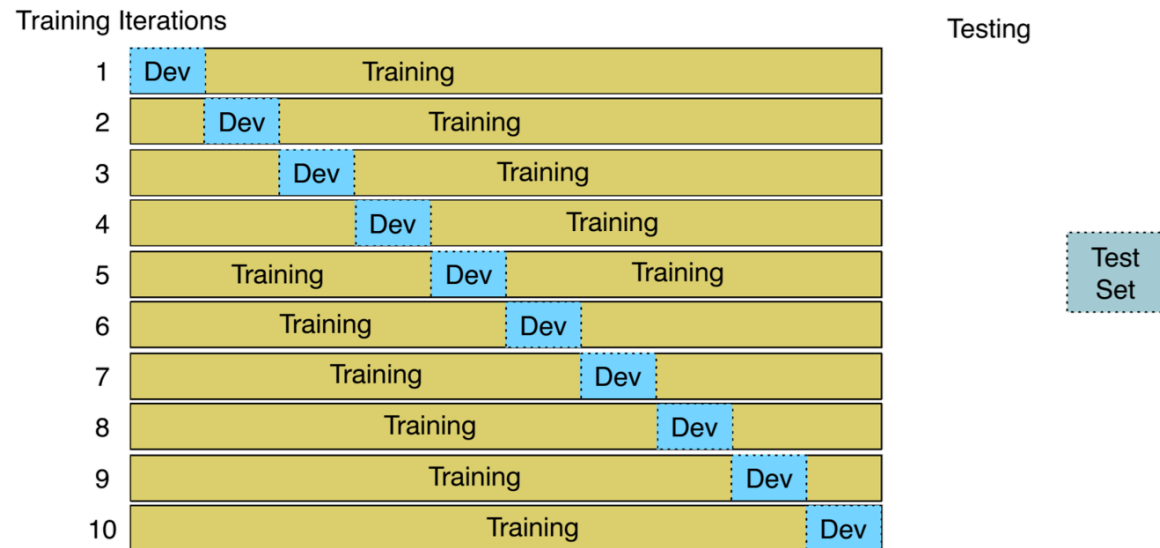
- Usually 80:20 or 70:30 splits
- Making sure the splits make sense
 - Time series: Split by time
 - i.i.d: Uniformly at random
 - Make sure you don't split the same datapoint between D_{TR} and D_{TE}
 - Make sure the same data does not get repeated on both sides e.g. spam
- We never look at the test data
- We train only using the D_{TR} and only use the D_{TE} once
- Then the test error approximates the generalization loss
- **How do we evaluate the model, if we do not have access to the test data while training?**

Validation Set



- E.g. Split 80:10:10
 - Train on D_{TR} , tune parameters or calculate error on D_{VA} , and finally test once on D_{TE}

- Cross-validation:



- Finally, train on the whole data once, before shipping out

Summary

- **Learning**

$$h^* = \operatorname{argmin}_{h^* \in H} \frac{1}{|D_{TR}|} \sum_{(x, y) \in D_{TR}} \ell(x, y, h)$$

- **Evaluation**

$$\epsilon_{TE} = \frac{1}{|D_{TE}|} \sum_{(x, y) \in D_{TR}} \ell(x, y, h^*)$$

- If the samples are drawn i.i.d. from the same distribution P , then the testing loss is an unbiased estimator of the true generalization loss

- **Generalization**

$$\epsilon = E_{(x, y) \sim P} [l(x, y) | h^*]$$

- $\epsilon_{TE} \rightarrow \epsilon$ as $|D_{TE}| \rightarrow +\infty$
- This is due to the weak law of large numbers, which says that the empirical average of data drawn from a distribution converges to its mean.

Thank You